INDEX

NAME: A. Sahana STD.: _____ SEC.: _____ ROLL NO.: _____ SUB.: _____

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.	22/7/26	Study of Network commands.		28/7/26 (9)
2.	27/7/26	Study of different types of network cables.		4/8/26 (9)
3.	04/08/26	Cisco packet Tracer.	(9)	4/8
4.	10/8/26	Wireless LAN	(9)	10/8
5.	18/8/26	Wireshark.	(9)	18/8
6.	7/9/26	Error detection & correction using hamming code	(9)	7/9
7.	22/09/26	Nmap to discover live host using bypassing platform	(9)	22/9
8.	28/9/26	End to End Communication at transport layer.	(9)	28/9
9.	28/9/26	Sliding Window protocol	(9)	28/9
10.	6/10/26	Subnetting	(9)	6/10
11	6/10/26	PING program	(9)	6/10

GKC0823316 B

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
12	12/10/26	Raw sockets to implement packet sniffing	9	12/10/26
13	12/10/26	Routing at network layer	9	21/10/26
14.	12/10/26	Inter networking with routers.	9	21/10/26

22/11/24 EXP 1 STUDY OF NETWORK COMMANDS

Aim:

To study of various network commands used in Linux and Windows.

arp -a:

ARP is short form of address resolution protocol. it will show the Ip address of your computer along with the Ip address and MAC address of your router.

Hostname:

This is the simplest of all TCP/Ip commands it simply, display the name of your Computer.

ipconfig/all:

This commands displays detailed configuration information about your TCP/Ip connection including Router, Gateway, DNS DHCP and type of Ethernet adapter in your System.

nbtstat -a:-

This commands helps solve problems with net BIOS name Resolution. (NBT Stands for NET BIOS over TCP/Ip)

Netstat:

netstat displays a variety of statistics about a computer active TCP/Ip connections both Incoming and outgoing as viewing routing tables.

nslookup :-

(name server lookup) is a tool used to perform DNS lookups in linux. It is used to display DNS details, such as the IP address of a particular computer.

eg: nslookup www.google.com.

pathping:

pathping is unique to windows and is basically a combinations of the ping and commands. pathping traces route to the destination address then branches as os second of each router along the way gathering statistics on the rate of datalom along each

ping:

(packet + internet grapher) command is the best way to test connectivity between two nodes. ping use ICMP to communicate to other device.

1) # ping hostname (ping localhost)
2) # Ping IPaddress (ping 4.2.2.2)
3) # ping fully qualified domain name.

(ping www.facebook.com)

Route:

Route command is used to show/ manipulate the IP routing table it is primarily used to setup static routes to specific host or networks via an interface.

Some Important Linux networking Commd

1) Ip

The Ip commands is one of the basic commands any administration will used in daily work. from setting up new systems and assigning IPs to troubleshooting existing systems and amongly Ip to trouble shooting existing system.

Ip <options> <OBJECT> <commands>

a) to show the Ip addrs to an Interface on your server.

a) [root@server ~]# Ipaddrs show

b) to assign Ip to an Interface, for example.

eps 03:

a[root@server ~] #Ip addrs add 192.168.1.254/24 dev inps 03

→ to delete an Ip on an Interface:

a. [root@server ~] # Ipaddrs 192.168.1.254/24 dev inps 03

d) After the status of the interface by bringing the Interface into.

e) After the status of the Interface. by bringing the interface into offline: [root@server ~] # Ip link set down.

g) add a default route (for all addres) via the local gateway 192.168.1254 that can be reached on device etho.
[root@sewa ~]# ip address route add default via 192.168.1.254 dev etho
h. add a route the 192.168.1.0124 via the gateway at 192.168.1.254;
[root@sewa ~]# ip route add 192.168.1.0126 via 192.168.1.24
i) add a route for 192.168.1.02124 via the gateway at that number reached on device.
etho: [root@sewa ~]# ip route add 192.168.1.0129 dev etho.
i) Delete the route for 192.168.1.024 via the gateway at 192.168.12.254.
K) display the route taken for Ip.10.10.1.4 [root@sewa ~]#
Ip route get 10.10.19

2) ifconfig:
The ifconfig commands was is a staple in many sysdmins tool belt for configuration and troubleshooting networks It has since been replaced by the Ip Commands discussed above.

3. mtr:-
mtr (matts traceroute) is program with a command line interface that.

server as a network diagonsit and troubleshooting tool. This command combines the function of the ping and traceroute commands. Just like a traceroute command will show the route from a computer to specified host mtr produce a lot of statistics about each hop such as response time and percentage. with the mtr commands you will get more information about route and be able to see problematic devices along the way if you see a sudden increase in response time or the syntax of the commands is as follows mtr <options> hostname/ip.

a) The bask mtr commands show you the statistics including each hop (hostname) with time and loss %.
[root@server ~] # mtr google.com.

b) Show numeric Ipaddress if you use. you will get Ip address (numby) instead of hostname)
[root@server ~] # mtr -g google.com

c) Show the numeric Ip address and host names
[root@server ~] # mtr - google.com.

d) Set the number of pings that you want to send: [root@server ~]#
mtr -c 10 google.com.

4) tcpdump:
The tcpdump commands is designed.

you can install tcp dump with the command below:-

[root@sewa ~]# dnf install -y tcpdump

before starting any capturing you need to know which interface can you will need to used sudu or host root acces in this case [root@sewer ~]# tcpdump -i)

if you want to capture traffic on eth0 you can intiate that with tcpdump -i eth0

sample output:

[root@sewa ~]# tcpdump -i eth0

[root@sewer ~]# tcpdump -i eth0 -co.

capture traffic to and from one host you can filter out traffic coming from specifically. for example I to find traffic coming from and going to 8.8.8.8.

use the command:

[root@sewa ~] tcpdump -i eth0 c lohost 8.8.8.8.

for traffic comming from 8.8.8 use:

[root@sewa ~]# tcpdump -i eth0 host 8.8.8.8

capture traffic to and from a specific network using the command.

[root@sewa ~]# tcpdump -j eth0 net 10.100

255.255

Or:

[root@

cap

numb

[root

ho

To

[root

lohost

To

and 2

[roo

port

Result:

windo

and

255.255.255.0

Or:

[root@ server ~]# tcpdump -i eth0 net 10.10

capture traffic to and from port numbers capture only DNS port 53 traffic

[root@ server ~]# tcpdump -i eth0 host 8.8.8.8 and port 53

To capture only HTTPS traffic

[root@ server ~]# tcpdump -i eth0 -c 10 host www.google.com and port 443

To capture all port except port 80 and 25

[root@ server ~]# tcpdump -i eth0 port not 53 and not 25.

Result:- The above both linux and windows commands are executed and output are completed successfully.

27/8/26

27/7/26

Practical-2

Study of Different types of network cables

Different type of cables used in networking are

1) unshielded Twisted pair (UTP) cable
2) shielded twisted pair (STP) cable
3) coaxial cable
4) fibre optic cable -

Cable Type	category	Maximum Data transmission	Advantages/ Disadvantages	Application/ Use
UTP	category 3	10bps	Advantages → cheaper in cost → easy to install as they have smaller overall diameter	→ 10 Base T - ethernet → fast, ethernet, gigabit ethernet
	category 5	upto 100Mbps		
	category 5e	1Gbps		
			Disadvantages: → more prone to (EMI) electro-magnetic interference & noise.	
STP	category 6, 6a	10Gbps	Advantages:- → shielded → faster than UTP. → less susceptible to noise & interference	gigabit ethernet, 10G ethernet (55m) widely used in data centers
SSTP	Category 7	10Gbps	Disadvantages → expensive → Greater installation effort	Gigabit ethernet, 10G ethernet (100m)

Cable Type	category
Co-axial cable	RG-6 RG-5 RG-11
Fibre optics cable	Sing mo Mul Mo

Result:-
The
cable

7/8/26

Cable type	category	Maximum Data transmission	Advantages/ Disadvantages	Applications ex.
Co-axial cable	RG-6 RG-59 RG-11	10-100 Mbps	Advantages: → High bandwidth → Immune to interference → low loss band-width → versatile Disadvantages → limited distance → cost → Size is bulky	Speed of signal is 500m Television network High speed Internet connections
Fibre optics cable	Single mode Multi mode	100 Gbps	Advantages: → High speed → High bandwidth → High security → long distance Disadvantages → expensive → Requires skilled installers.	Maximum distance of fibre optics cable is around 100 metres

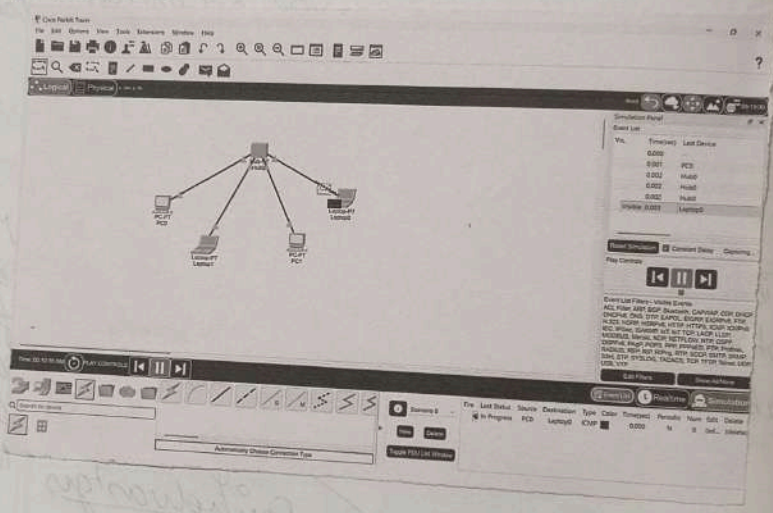
Result:-
Thus the diffrent types of network cables are studied Succesfully.

7/8/24

Exp: 3 Experiments on cisco packet Tra
(simulation tool)

Aim:- To understand its environmen
and behaviour of network devices
Cisco packet tracer Simulator.

1) peer-to peer connection (P2P):-

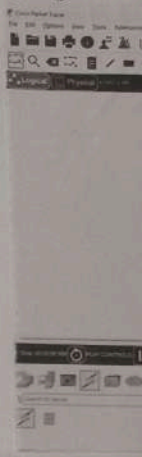


cross-over cables were used to connect the PC and the laptop.

only a cross-over cable can enables communication directly between same type of end-devices.

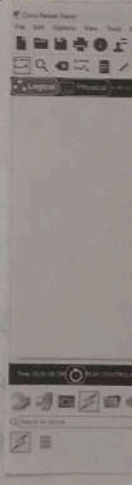
Usage of such cables have been replaced by switches, hubs and Routers.

2) Hub conn



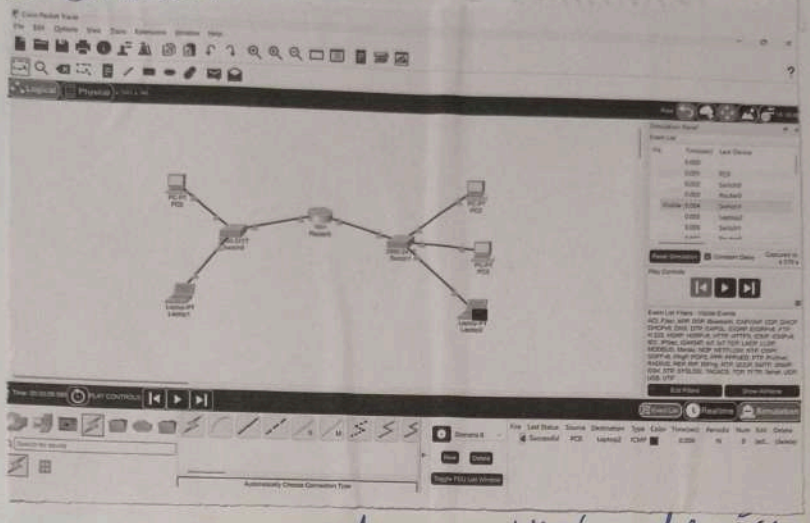
A hub
network
incoming
connected

3) Switch C



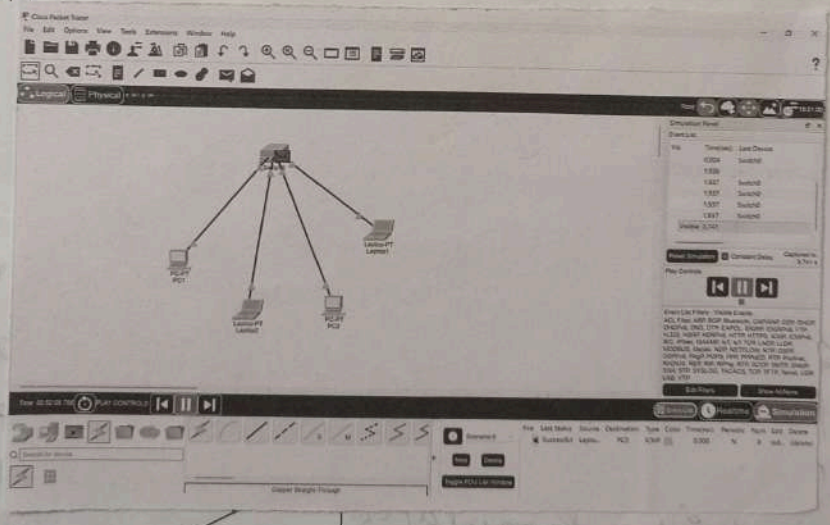
Switch co
LAN and
directly
MAC ad

2) Hub Connection:-



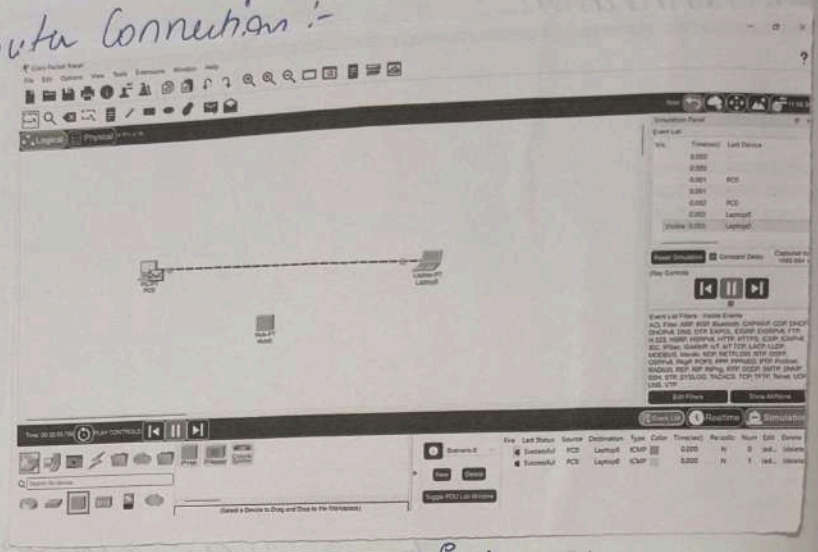
A hub connects multiple devices in a network by blindly broadcasting all incoming data signals to every connected device.

3) Switch Connection:-



Switch connects multiple devices within single LAN and intelligently forwards data packets directly to specific destination using MAC addresses.

4) Router Connection :-



A router connects different subnets or distinct networks together, directing data traffic between them using IP addresses and default gateways.

Result :- Hence the environment of Cisco packet tracer and behaviour of all the network devices in the software was studied. Simulated and verified Successfully.

9/8/25

Exp: 4. Setup and Configure a LAN using Switch and Ethernet Cables.

What is LAN?

LAN stands for Local Area Network which is a group of computers and connected devices linked together in a small physical location like a home, school or office building.

How to set up LAN.

Step 1 :- Connect the Hardware
→ place your router in a central location near a power outlet.
→ Connect your modem to the router's WAN or internet port using an ethernet cable
→ plug the router into power and wait for it to turn on

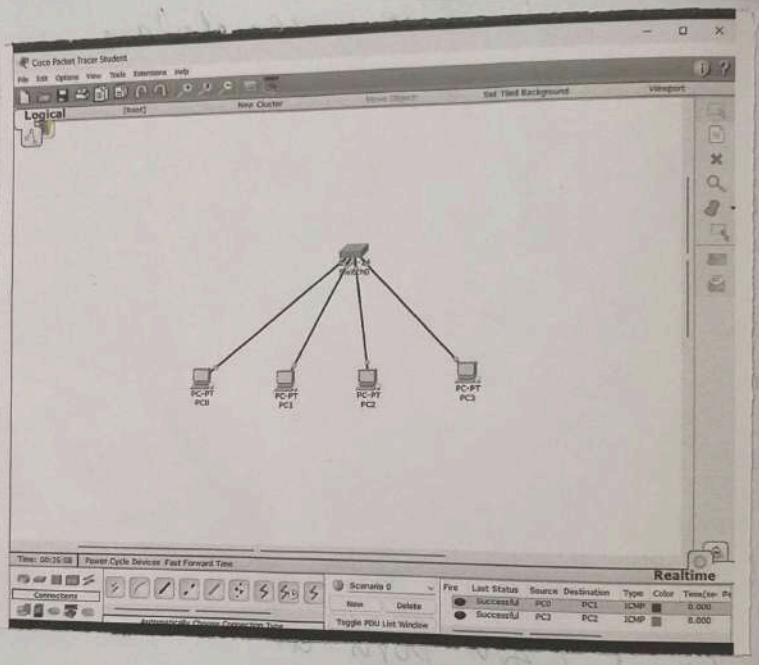
Step 2 :- Connect your devices
→ Wired setup : plug one end of an ethernet cable into your computer's network port and the other end into one of the numbered LAN ports on the router

Step 3 : Configure the router
→ Type your IP address (192.168.1.1 or 192.162.0.1) into the address bar and press enter.
→ login using the default username and password printed on the router.

Step 4 :- check the connectivity between Switch and other machine by using ping command in the command prompt of the device.

Steps:-

Try to acces the shared folde... from others computers of the Netwo...



Result:-

Wireless LAN using switch and ethernet cables are executed successfully as also packet traces.

18/08/26 Exp...

Aim:- Ex... Wire... sh... To filte... wireshar... The Ethe... and Sa...

Procedure:

* Select...
* To C...
* Selec... after l...
* Then ...
* Searc...
* To se... graph...
* Save ...

1. Creat... packets... the fl...
* Sele...
* TO ...
* Selec... after ...
* Then ...
* Sav...

2. Creat... packets... procedu...
* G...
* S... afte...

18/08/26 Exp 5 WIRESHARK

Aim:- Experiments on packet capture tool: Wireshark
To filter, capture, view, packets in wireshark tool capture 100 packets from The Ethernet: IEEE 802.3 LAN Interface and save it.

Procedure:-
* Select local Area Connection in Wireshark
* To capture option
* Select stop capture automatically after 100 packets.
* Then click Short capture.
* Search Tcp packets in Search bar
* To see flow graph click statistics flow graph
* Save the packets.

1. Creat a Filter to display only TCP/UDP packets, inspect the packets and provide the flow graph.
* Select LAN connection in Wireshark
* To capture option
* Select stop capture automatically after 100 packets
* Then click start capture
* Save the packets

2. Creat a filter to display only ARP packets and inspect the packets

procedure:-
* GTO to capture option
* Select Stop capture automatically after 100 packets

* Then click start capture
* Search ARP packets in search bar
* Save the packets.

3. Create a filter to display only HTTP packets and inspect the packets

procedure.

* Select Local Area Connection in Wireshark
* Go to Capture option
* Select Stop Capture automatically after 100 packets
* Then click start capture
* Search HTTP packets in search bar.

4. Create a file to display only DNS packets and provide the flow graph

procedure:-

* Go to Capture option
* Select Stop Capture automatically after 100 packets
* Then click Start Capture
* Search DNS packets in search bar
* To see the flow graph click Statistics flow graph.

5. Create a filter to display only DNS packets and inspect the packets

18/8/22

* Select local area connection in wireshark
* Go to capture option
* Select stop capture automatically after 100 packets
* Then click start capture
* Search DHCP packets in search bar
* Save the packets

Result:-

Wireshark on packet capture tool experiment are learned and executed successfully.

18/8/22

7/9/26 Exp: 6 Error detection & correction using Hamming code.

Aim:- Write a program to implement error detection and correction using hamming code and concept Make a test run to input data stream and Verify error correction feature.

Sender:-
* input to sender file should be a bit of any length. program should be convert the text to binary
* Apply hamming code concept on the binary data and add redundant bits to it.
* Save this output in a file called channel.

Code:

```
data = input ("Enter 4 bit:")
d1, d2, d3, d4 = map (int, data)
P1 = d1 ^ d2 ^ d4
P2 = d1 ^ d3 ^ d4
P4 = d2 ^ d3 ^ d4
code = [P1, P2, d1, P4, d2, d3, d4]
print("Sender Hamming code", "".join
      (map (str, code)))
```

Receiver side:-

* Receive the hamming code from the sender.
* Enter the received code
* Calculate the parity check values
* Find the error position using the parity values
* Correct the error by changing the bit
* Display the corrected hamming code

Code:-

```
r = list(map(int, input("Enter received 7 bits:")))
r1 = r[0] ^ r[2] ^ r[4] ^ r[6]
r2 = r[1] ^ r[2] ^ r[5] ^ r[6]
r3 = r[3] ^ r[4] ^ r[5] ^ r[6]
error = r1 * 1 + r2 * 2 + r4 * 4
if error == 0:
    print("No error")
else:
    print("Error at position:", error)
    r[error - 1] ^= 1
    print("Corrected code: ", "".join(map(str, received)))
```

Input:-

Enter 4 data bits : 1011
Sender Hamming code : 0110011
Received 7 bits : 0010011

output

Error at position : 2

Corrected code : 0110011

Sample Input 2 :-

Enter 4 data bits : 0010

Sender Hamming code : 0101010

Enter received 7 bit : 0101010

output :-

No Error.

Result :-

Error detection & error correction using hamming code was executed successfully.

22/09/26

Exp
Uni
platf

Aim :-

To
Scans
Try Ha
Try h
proced

Ta
* A
requ
* I
to id
* T
pack
data

Task
Ne
of Co
a

Subn
equi
Sym
Is e
IP

*

*

22/09/26 Exp: 7 NMAP to Discover Live hosts using Nmap Scans On the TryHackMe platform.

Aim:-

To Discover live Hosts using Nmap Scans (ARP, ICMP, TCP/UDP) on the TryHackMe platform Room link; https://tryhackme.com/room/nmapol.

procedure:-

Task 1:- Introduction.

* ARP scan: This scan uses ARP request to discover live hosts
* ICMP Scan: This scan ICMP request to identify live hosts.
* TCP/UDP ping scan: This scan sends packets to TCP ports and UDP ports to determine Live hosts.

Task 2:- Subnetworks.

Network segment - It is a collection of Computers linked together through a common "Medium".

Subnetworks:- It is usually the equivalent of one or more networks segments linked together and configured to utilize the same router in an IP networks.

* Subnets with /16
* Subnets with /24.

Task 3:- Enumerating Targets.

We need to identify the target we want to scan before we conduct any "action"

1) list
- <MACHINE-IP> <scanme.nmap.org> <example>
- will scan 3 IP address.

2) range
- 10.11.12.15-20
- will scan 6 IP addresses.

3) subnet
- <MACHINE-IP>/30
- will scan 4 IP addresses

nmap -sL <TARGETS>

Task 4:- Discovering like Hosts.

1) ARP from link layer - commonly used for Man-in-middle-Attack (ARP Spoofing and ARP poisoning)

2) ICMP from network layer - Commonly used in DDOS attacks.

Task 5:- Nmap Host Discovery using ARP

1) When a privileged user tries to scan LAN (local area network), nmap uses ARP requests

2) To discover, internet hosts without port scanning - like systems are nmap -sn <target>.

Task 6:- Nm

1) Add -
echo req

2) nmap
host is

3) IP · op
ICMP hi

Task 7:- Nm
TCP and

1) Nmap
followed
or a com

- PS21,
29, 25.

2) nmap
TCP por

3) TCP
nmap as

4) nmap
the onlin

5) UDP p
to an op
in a re

6) If we
closed UD
receive a

Task6:- Nmap host discovery using ICMP.

1) Add -PE option to enable ICMP echo requests to discover live hosts.

2) nmap -PE -sn Target to check host is live.

3) -PP option instructs Nmap to use ICMP timestamp requests.

Task7:- Nmap Host discovery using TCP and UDP.

1) Nmap to use TCP SYN ping use -PS followed by the port number, range, list or a combination of this options.

-PS21, -PS25 targets ports 21, 22, 23, 24, 25.

2) nmap -PS -sn TARGET check the TCP port is open.

3) TCP ACK ping - you must be running nmap as a privileged user.

4) nmap -PA -sn TARGET to discover the online hosts on the target's subnet

5) UDP ping - sending a UDP packet to an open port is unlikely to result in a response.

6) If we transmit a UDP packet to a closed UDP port, we should expect to receive an ICMP port unreachable packet

Task :- Using Reverse - DNS lookup.

1) Nmap is to use reverse - DNS internet hosts
2) -R option to query the DNS ser even for offline hosts

Task 9 :- Summary.

ARP Scan, ICMP Echo Scan, ICMP Timestamp Scan, ICMP address mask scan, TCP SYN ping scan, TCP ACK ping scan, UDP ping scan.

Results :-

NMAP to discover live host using Pythonmap platform was successfully executed.

28/9/26 Exp 8

a) Aim :-
using
procedu
* Cr
progra
* Spe
numbe
* Bind
addre
* MAC
Conn
* Conne
* Displ
* Clos

Code :-
TCP-
import
Serve
So cr
Serve
Serve
print(
Conn,
print(
while T
data

28/9/26 Exp 8 :- End to End Communication at Transport layer.

a) Aim :- Implement echo client-server using TCP/UDP packets.

Procedure :-

* Create a socket using python socket programming
* Specify the IP address and port number
* Bind the server socket to the IP address and port.
* Make the server listen for client connections.
* Connect the client to the server.
* Display the received message.
* Close the socket connection.

Code :-

TCP - Server.

```
import socket
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind("127.0.0.1", 5000)
server.listen()
print("server is waiting for connection")
conn, addr = server.accept()
print("client connected")
while True:
    data = conn.recv(1024).decode()
```

```
    if not data:
        break
    print("client:", data)
    conn.send(data.encode())
conn.close()
server.close()
```

TCP - Receiver (client)

```
import socket
client = socket.socket(socket.AF_INET,
socket.sock_stream)
client.connect(("127.0.0.1", 5000))
message = input("Enter message:")
client.send(message.encode())
reply = client.recv(1024).decode()
print("server:", reply)
client.close()
```

Terminal 1 - Server

```
Python tcp-server.py
Server is waiting
for connection...
client connected
Client: Hello.
```

Terminal 2 - client

```
Python tcp-client.py
Enter message: Hello
Server: Hello
TCP echo is Working
```

Result:

The TCP/UDP packets using echo client server Socket program is executed successfully

28/9/25

28/9/25 Ex9:

Aim:- Wri

control
window
one no

Code:-

imp
win
mes
Sta
whi
pri
end
if

#Se
Sno
for

#Del
#Rec

28/9/26 EX9: SLIDING WINDOW PROTOCOL.

Aim:- Write a program to implement flow control at data link layer using sliding window protocol simulate the flows frame one node to another.

Code:-

```
import time
window = int(input("Enter window size: "))
message = input("Enter message: ")
start = 0
while start < len(message)
print("SENDER")
end = start + window
if end > len(message):
    end = len(message)
#Sender buffer
Sender_buffer = []
for i in range(start, end):
    frame = (i, message[i])
    Sender_buffer.append(frame)
    print("Frame no:", "DATA:", message[i])
print("\n Sender Buffer: ")
print(Sender_buffer)
#Delay
print("\n Waiting for receiving...")
time.sleep(1)
#Receiver
print("Receiver")
expected = start
error = false
for frame in Sender_buffer:
```

```
frame_no = frame[0]
data = frame[1]
print("Received frame:", frame_no, "Data:", data)
if frame_no == expected:
    expected = expected + 1
else:
    error = True
    break
# Receiver buffer
if error:
    receiver_buffer = ("NACK", expected)
    print("\n Receiver Buffer:")
    print(receiver_buffer)
else:
    ack = expected - 1
    receiver_buffer = ("ACK", ack)
    print("\n Receiver buffer:")
    print("receiver_buffer")
# Sender reads ACK/NACK
print("\n SENDER CHECKING RESPONSE")
response = receiver_buffer[0]
number = receiver_buffer[1]
if response == "ACK":
    print("ACK received:", number)
    start = number + 1
else:
    print("NACK received:", number)
    start = number
print("Transmission completed")
```

OUTPUT

Ente
Ente
SENDE
Frame n
Frame no
Frame no
Sender Bu
[(0, 'H'),
waiting
RECE
Received
Received
Received
Receiver
('ACK',
SENDER
ACK RE

Result

28/1/21

sl
and

OUTPUT :-

Enter window size: 3
Enter menag: Hello.

SENDER

Frame NO: 0 DATA: H
Frame NO: 1 DATA: e
Frame NO: 2 DATA: l.
Sender Buffu
[(0,'H'), (1,'e'), (2,'l')]
waiting for receiv....

SENDER

Frame no: 3 DATA: l
Frame no: 4 Data: o
sender Buffu:
[(3,'l'), (4,'o')]
waiting for receiv....

RECEIVER

Received Frame: 0 DATA: H
Received Frame: 1 DATA: e
Received Frame: 2 DATA: l
Receiver Buffu:
('ACK', 2)
SENDER CHECKING RESPONSE
ACK RECEIVED: 2

RECEIVER

Received frame: 3 DATA: l
Received Frame: 4 DATA: o
Receiver Buffu:
('ACK', 4)
SENDER CHECKING RESPONSE
ACK Received: 4

TRANSMISSION COMPLETED.

Result

Sliding window protocols are learned and executed successfully.

28/11/21

6/10/25 Exp10 Subnetting in cisco packet Trace

Aim:-
To implement classless IP Subnetting using cisco packet tracer and verify connecting between different Subnets.

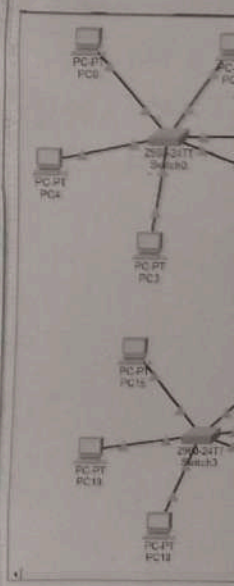
Requirement:-
* cisco packet Tracer
* Routers
* Switches
* PCs
* Ethernet Cables.

Procedure:-
1. open cisco packet Tracer.
2. Create a new network topology.
3. Add routers, Switches and PCs to the workspace. connect the device using ethernet cables.
4. Divide the network into Smaller Subnets using the /27 Subnet mask.
5. Configure the IP address and Subnet mask of the router Interfaces.
6. Configure the required Switch ports
8. Enable the router Interfaces using no shutdown command.
9. check the ip configuration of all devices the ping command to check connectivity between PCs

IP Addr

PC-PT PC
PC-PT PC1
PC-PT PC2
PC-PT PC3
PC-PT PC4
PC-PT PC5
PC-PT PC6
PC-PT PC7

2960-24T
Switch1
2960-24
Switch2



Result:-
Thus
implemented
connectivity
wired

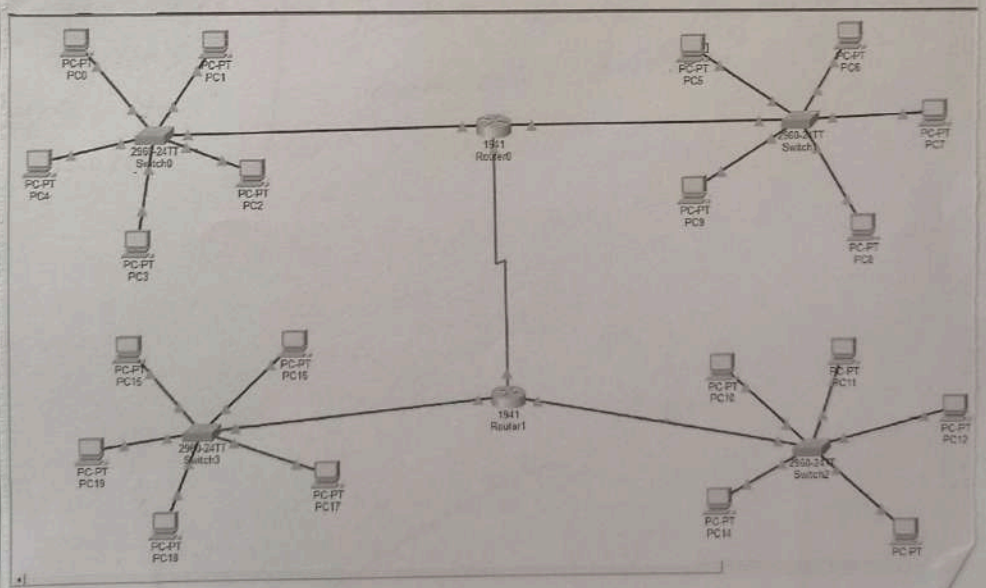
6/10/25

Ip Addressing

	Ipaddress	Subnetmask	Default gateway
PC-PT PC0	192.168.2.12	255.255.255.192	192.168.3.1
PC-PT PC1	192.168.2.11	255.255.255.192	192.168.3.1
PC-PT PC2	192.168.1.12	255.255.255.192	192.168.1.1
PC-PT PC3	192.168.1.11	255.255.255.192	192.168.1.1
PC-PT PC4	192.168.2.14	255.255.255.192	192.168.4.1
PC-PT PC5	192.168.2.13	255.255.255.192	192.168.4.1
PC-PT PC6	192.168.1.14	255.255.255.192	192.168.2.1
PC-PT PC7	192.168.1.13	255.255.255.192	192.168.2.1

2960-24TT Switch1, 2960-24TT Switch2 → These four switches used to connect with PCs and routers.

2901 used to communicate



Result:-

Thus Classless Ip Subnetting was successfully implemented using cisco packet tracer and connectivity between the network devices was verified successfully.

6/10/26 Exp 11 PING PROGRAM

Aim:-

To implement a ping program using socket programming to check the connectivity between two network devices

Program:-

```
import socket
import time
import sys
def udp_ping_server(host="127.0.0.1", port=
    server_socket = socket.socket(socket.AF_
                                  SOCK_DGRAM)

    server_socket.bind((host, port))
    print(f"[Server] Listening on {host}:{port}
    while True:
        msg, client_address = server_socket.recvfrom
                                  (1024)

        print(f"[SERVER] Received {msg.decode
            from {client_address}")
        server_socket.sendto(msg, client_addr
try:
    data, _ = client_socket.recvfrom(1024)
    end = time.time()
    rtt = (end - start) * 1000
    print(
```

output:

```
pytho
[SER
[SERV
6564
[SERV
fro
Python
Repl
6564
```

6/10/26 Result

```
f" Reply from {sever_host}: {sewa_port}"
"f" {data.decode()} | RTT = {rtt:.2f}ms")
except socket.timeout:
    print(f" Request {i} timed out")
client_socket.close()
if __name__ == "__main__":
    if len(sys.argv)>1 and sys.argv[1]==
    "sewa":
        udp_ping_sewa()
    cln:
        udp_ping_client()
```

Output:

```
python udp_ping.py sewa.
[SERVER] Listening on 127.0.0.1:12000
[SERVER] Received 'ping | 1791476473.
6564088' from ('127.0.0.1', 58522)
[SERVER] Received 'ping 2 1791476483698525'
    from ('127-0-0.1', 58522).
python udp_ping.py
  Reply from 127.0.0.1:12000 ping | 1791476483
  6564088 | RTT = 0.92ms.
```

6/6/24 Result:

Thus, the ping program is successfully executed.

12/10/25 Exp 12 Write a code using RAW sockets to implement packet sniffing

Aim: To write a python program using RAW sockets to capture and analyze network packets.

Algorithm:

* Start the program
* Create a raw socket using SOCKET.AF_INET and SOCKET.SOCK_RAW
* Capture incoming network packets.
* Extract the Ethernet header from each packet.
* Identify the source MAC address, destination MAC address and ethernet protocol.
* Display the packet details
* Repeat until the program is stopped

CODE:

```
from scapy.all import sniff
from scapy.layers.inet import IP, TCP, UDP,
def packet_callback(packet):
    if IP in PACKET
        ip_layer = PACKET[IP]
        Protocol = IP_layer.Proto
        src_IP = IP_layer_SRC
        dest_ip = ip_layer_dest

    elis protocol == 17:
        Protocol_name = "UDP"
    else
        Protocol name == "Unknown protocol"
```

sniff (ifac

OUTPUT

Destination
Protocol : TC
Source IP :
Destination
Protocol : U
Source IP:
Destination

```
Router>enable
Router#show ip route
S    30.0.0.0/8 [20/0]
S    50.0.0.0/8 [10/0]
S    60.0.0.0/8 [1/0]

Router#
%LINEPROTO-5-UPDOWN:

%LINEPROTO-5-UPDOWN:

%LINEPROTO-5-UPDOWN:

Router#show ip route
S    30.0.0.0/8 [10/0]
S    50.0.0.0/8 [10/0]
S    60.0.0.0/8 [1/0]
```

RESULT:

This Th
successfully
programming
and received

12/10

```
print(f"Protocol : {protocol_name}")
print(f"Source IP : {src_ip}")
print("Destination IP : {dest_ip}")
print("_"*50)

sniff(iface="wifi", prn=Packet_Callback,
      filter="ip", store=0)
```

OUTPUT

Destination IP: 224.0.0.251
Protocol : TCP
Source IP: 172.16.11.15
Destination IP: 172.16.8.1
Protocol : UDP
Source IP: 172.16.11.136
Destination IP: 239.255.255.250.

```
Router>enable
Router#show ip route static
S    30.0.0.0/8 [20/0] via 40.0.0.2
S    50.0.0.0/8 [10/0] via 40.0.0.2
S    60.0.0.0/8 [1/0] via 40.0.0.2

Router#
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

Router#show ip route static
S    30.0.0.0/8 [10/0] via 20.0.0.2
S    50.0.0.0/8 [10/0] via 40.0.0.2
S    60.0.0.0/8 [1/0] via 40.0.0.2
```

RESULT:

This The python program was successfully executed to check the socket programming messages were successfully sent and received between client and server

12/10/24 Exp13 Routing at Network Layer.

a.) Simulate static routing protocol configu... using cisco packet Tracer

AIM: To simulate a static routing configure... using CISCO PACKET TRACER.

PROCEDURE:

* Open cisco packet Tracer and create the given network topology.
* Connect 3 routers and PC's using suita... cables.
* Configure IP addresses for routers and PC's
* Enable all routers interfaces using no shutdown.
* Configure static routers on-Router 0, Router 1, and Router 2 using IP mobile.
* Test Connectivity
* Disconnect the main link and verify that the backup route works.
* Thus, a static routing is success fully configured and verified.

```
Pinging 20.0.

Reply from 2
Reply from 2
Reply from 2
Reply from 2

Ping statist
    Packets:
Approximate
    Minimum

C:\>tracert

Tracing rout

  1    0 ms
  2   11 ms
  3   29 ms
  4   22 ms

Trace comple
```

RESULT:

Thus, s... configured ... between all ... successfully

12/10/24

```
Pinging 20.0.0.2 with 32 bytes of data:

Reply from 20.0.0.2: bytes=32 time=55ms TTL=125
Reply from 20.0.0.2: bytes=32 time=56ms TTL=125
Reply from 20.0.0.2: bytes=32 time=60ms TTL=125
Reply from 20.0.0.2: bytes=32 time=55ms TTL=125

Ping statistics for 20.0.0.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 55ms, Maximum = 60ms, Average = 56ms

C:\>tracert 20.0.0.2

Tracing route to 20.0.0.2 over a maximum of 30 hops:

  1    0 ms      0 ms      0 ms      10.0.0.1
  2    11 ms     8 ms      5 ms      192.168.1.250
  3    29 ms     26 ms     25 ms     192.168.1.245
  4    22 ms     28 ms     20 ms     20.0.0.2

Trace complete.
```

RESULT:

Thus, static routing was successfully configured in CISCO PACKET TRACER, connectivity between all Networks was verified successfully using Ping.

12/10/24

b) Simulate RIP using CISCO PACKET TRACER

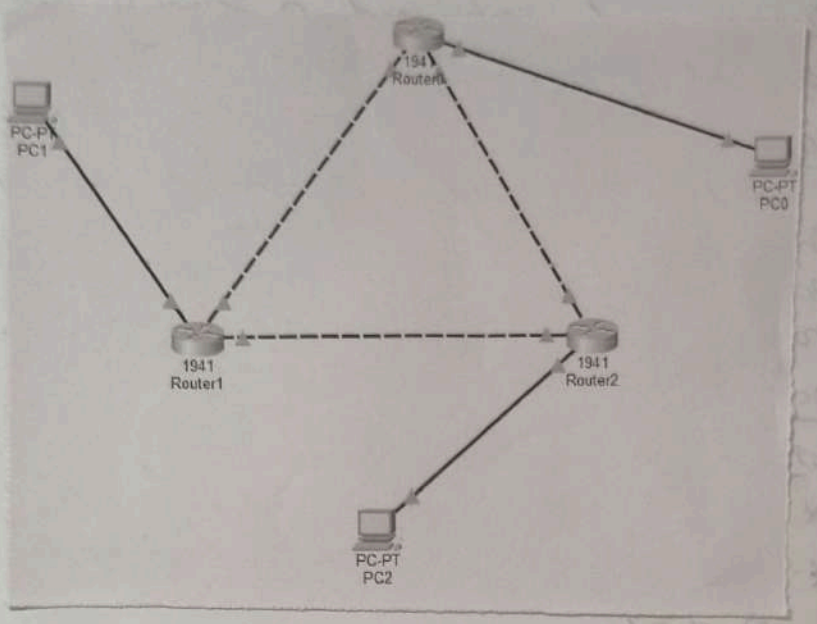
AIM:-
To simulate RIP using CISCO PACKET TRACER.

PROCEDURE:

* Create the topology with routers and and 2PCs
* Connect the device using suitable co
* Assign the given IP address to both PC
* Configure Fast Ethernet and serial interface of all routers.
* Enable interface using no shutdown.
* Set clock rate & bandwidth 64 on DC serial interface.
* Enable RIP on all routers using router RIP.
* Add directly connected networks using the network
* Test connectivity using ping 20.0.0.2
* Use trace 20.0.0.2 to verify the selected router.



12/12/24 Result:
RIP
routing



12/12/24 Result:

RIP successfully configured and dynamic routing between PC0 and PC1 was verified

tel col

Exp14: Internetworking with routers in CISCO PACKET TRACER simulator

a) Design and configured a simple internet using a router.

AIM:
To design and configure a simple internetwork using a router in CISCO PACKET TRACER and fast connectivity between two PCs

PROCEDURE:

Step 1: Configuring Router1
* Select the router and open [C].
* Press enter to start configuring Router1.
* Type enable to activate the privileged mode:

Step 2:
* Assign IP addresses to every pc in the network
* Select the PC, Go to the desktop and select IP configuration and assign an address, Default gateway subnet of PC0 as 192.168.10.1
* Assign the default gateway of PC1 as 192.168.2.01

Step 3:
* Connect fast ethernet 0 port PC0 with fast ethernet 0/0 port of Router1 using a copper straight-through cable.

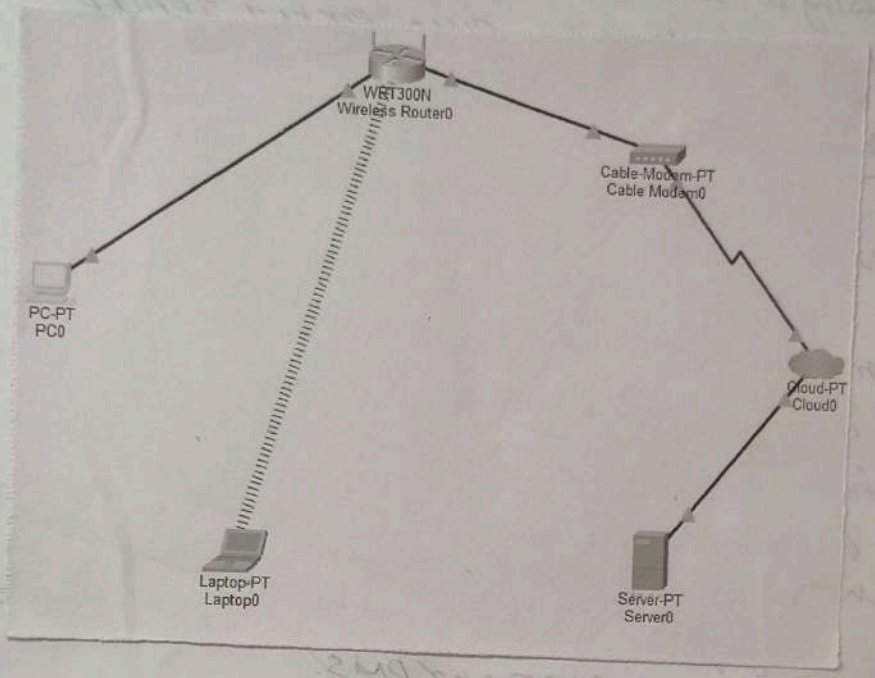
* Connect
fast ether
copper str



22/10/24

RESULT: The
successfully
a router
was succes

* Connect Fast ethernet 0 port of PC4 with fast ethernet 0/1 port of Router 1 using a copper straight through cable.



22/10/24

RESULT: The simple internetwork was successfully designed and configured using a router. connectivity between PC0 and PC4 was successfully verified using PDU.

b) Design and configure an inter network using wireless router DCMP and internet cloud.

AIM: To design and configure an internet using a wireless router, DHCP server and internet cloud in CISCO PACKET TRACER.

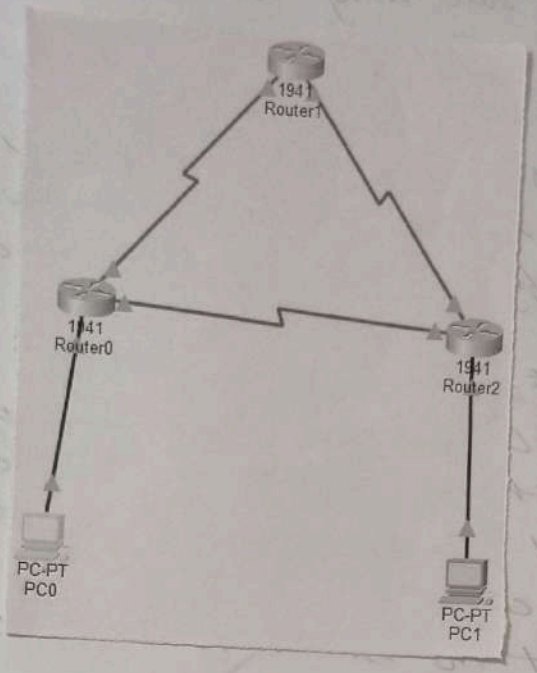
PROCEDURE:

* Open Cisco Packet Tracer and create the required network topology.
* Connect PC, laptop, wireless router, cable modem, internet cloud and server.
* configure the wireless router with SSID home and enable DHCP
* Connect the laptop to the wireless network and set the PC to DHCP.
* Configure the server with IP 208.67.220.2 and enable DHCP and DNS.
* Verify connectivity using IP config and ping cisco.com.

22/10/22

RESULT:

The int
designed an
router, DHCP
and conne
using ping



29/10/23

RESULT:

The internetwork was successfully designed and configured using a wireless router, DHCP server and internet cloud and connectivity was successfully verified. using ping cisco.com.

c.) Design and configure an internetwork in your lab using switch, router and Ethernet cables.

AIM:
To design and configure an internetwork using a switch, router and ethernet cables and verify connectivity between computers.

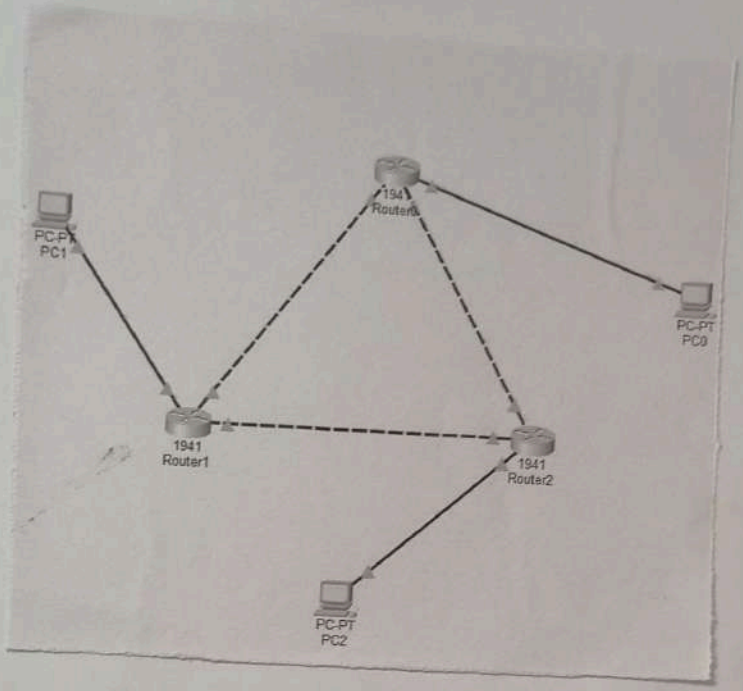
PROCEDURE:

* Open Cisco packet Tracer and place compu a switch and router in the workspace.
* Connect the computers to the switch us ethernet cables.
* Connect the switch to the router using a ethernet cable.
* Assign static IP addresses, subnet masks and default gateway addresses to computers.
* Configure the router interfaces with appropriate IP addresses and enable the interfaces.
* use the ping command to test connect between computers.
* Observe the successful transmission of packets.



RESULT:
Thus successfully a switch cisco packet

24/12/25



RESULT:

Thus an internetwork was successfully designed and configured using a switch, routers and ethernet cables in cisco packet Tracer.

24/2/24